Smart Resume and Job Matching System

University of Warwick
CS350 - Data Science Project
Michael Ionita

Presentation Outline

- Motivation & Project Objective
- Matching Model
- Web Development
- Demo
- Limitations & Future Work
- Project Management
- Q&A

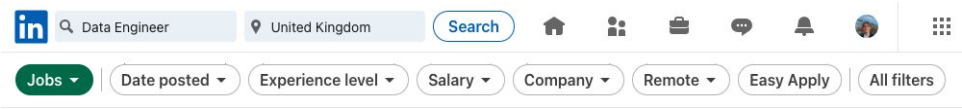
Section Outline

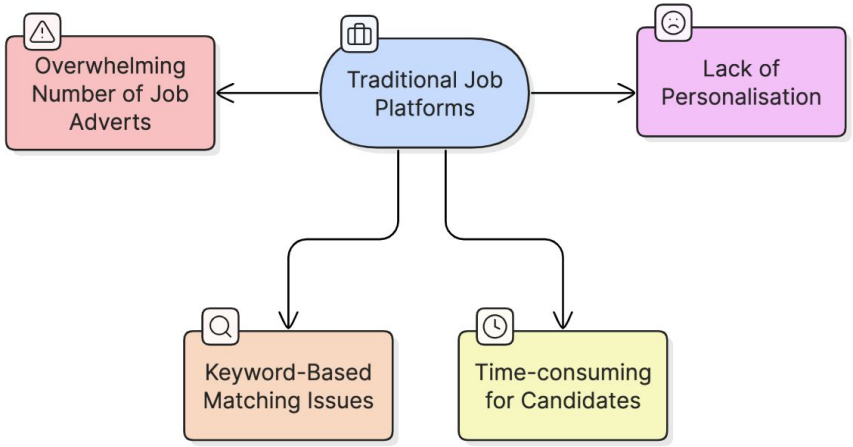
- **Motivation & Project Objective**
  - Motivation
  - Background & Research
  - Project Objective
  - High-Level Architecture
- Matching Models
- Web Development
- Demo
- Limitations & Future Work
- Project Management
- Q&A

Motivation

Challenges in Traditional Job Platforms

- Huge number of job adverts available (e.g., LinkedIn with **14M+ postings**)
- Keyword matching leads to irrelevant results & missed nuances
- Job seekers spend **11 hours** a week searching for jobs (*CareerBuilder*)
- Job seekers face frustration (e.g., filters, different job boards)





Background & Research

- **Effectiveness of Embedding-Based Matching**
  - Semantic embeddings outperform traditional keyword-based job matching
  - Capture nuanced relationships beyond simple keyword overlap
- **Identified Gaps**
  - Limited personalisation for technical skills
  - Lack of unified approach combining contextual and structured insights.
  - Manual checks still required for accurate matches

Embedding-based Recommender System for Job to Candidate Matching on Scale

Jing Zhao, Jingya Wang, Madhav Sigdel, Bopeng Zhang, Phuong Hoang, Mengshu Liu and Mohammed Korayem
CareerBuilder
Peachtree Corners, Georgia, US
{jing.zhao,jingya.wang,madhav.sigdel,bopeng.zhang,phuong.hoang,mengshu.liu,mohammed.korayem}@careerbuilder.com

Effectiveness of job title based embeddings on résumé to job ad recommendation

Mesut Kaya
mkaya@hum.aau.dk
Aalborg University Copenhagen
Copenhagen, Denmark

Toine Bogers
toine@hum.aau.dk
Aalborg University Copenhagen
Copenhagen, Denmark

An Intelligent Job Recommendation System based on Semantic Embeddings and Machine Learning

Priyanka Singla, Vishal Verma
Ch. Ranbir Singh University
Jind, India.
singla.priyanka198@gmail.com, vishal.verma@crsu.ac.in

Project Objective

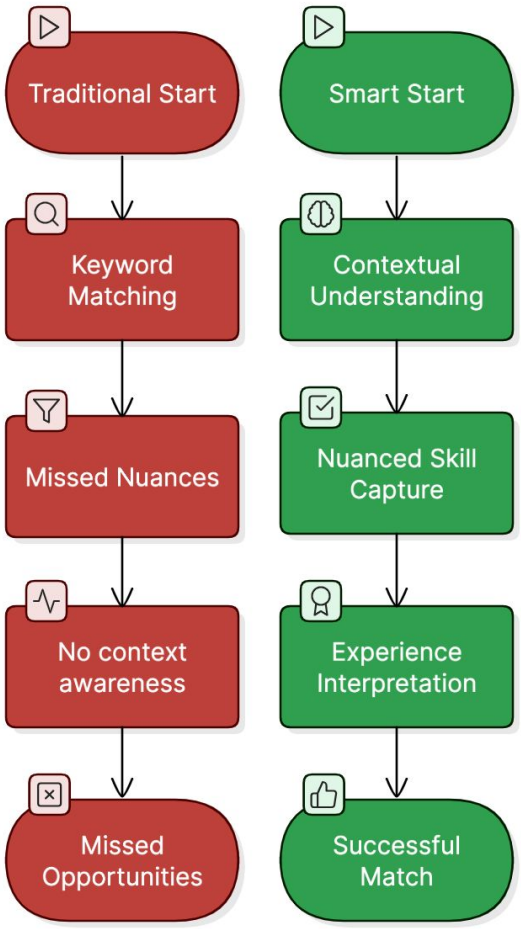
Goal: Develop a web-based platform to intelligently match candidates with job opportunities.

Benefits:

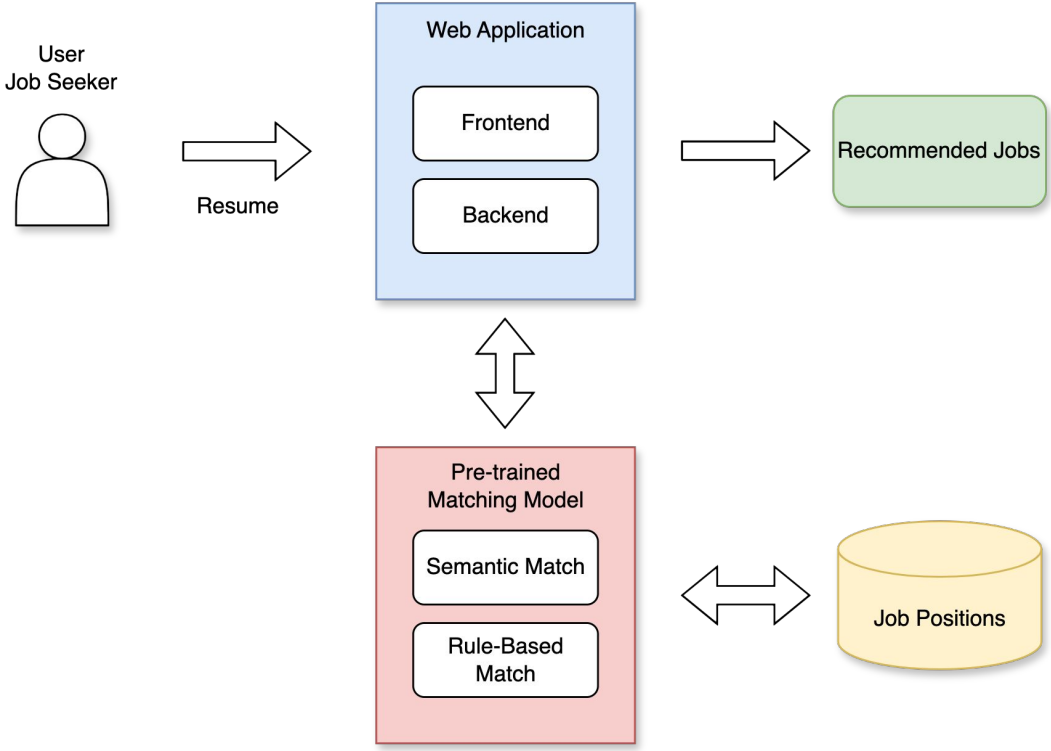
- Considers relevant details (skills, experience)
- Captures context, not just keywords
- Enhances matching accuracy
- Reduced job search time

Implementation:

- Matching Model (ML and NLP)
- Web Application (Frontend and Backend)



High-Level Application Architecture

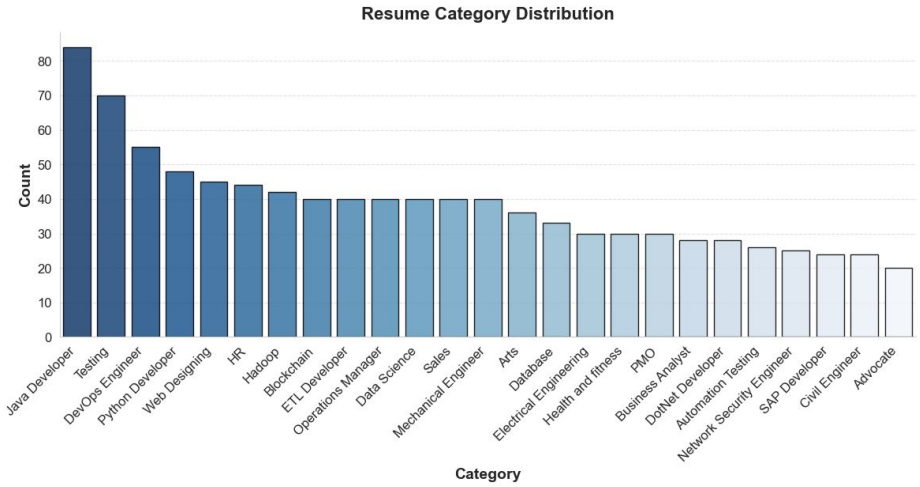


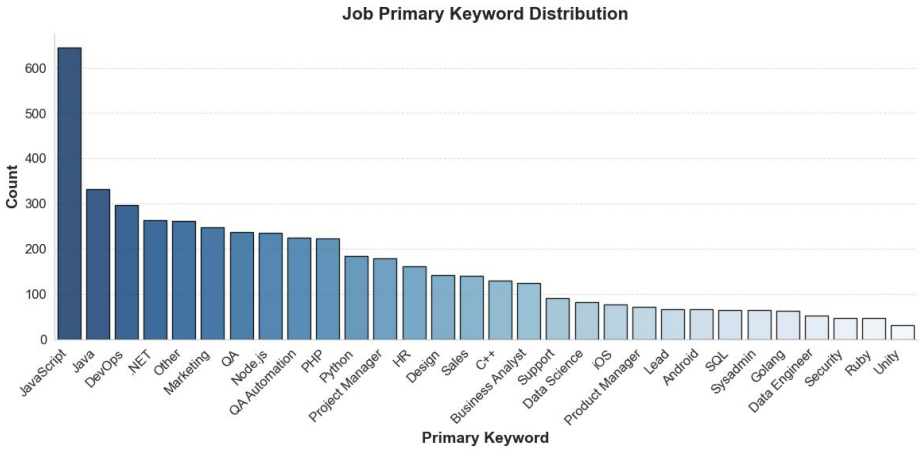
Section Outline

- ✓ Motivation & Project Objective
- **Matching Model**
  - Data Overview
  - Data Preprocessing
  - Matching Approaches
  - Evaluation
- Web Development
- Demo
- Limitations & Future Work
- Project Management
- Q&A

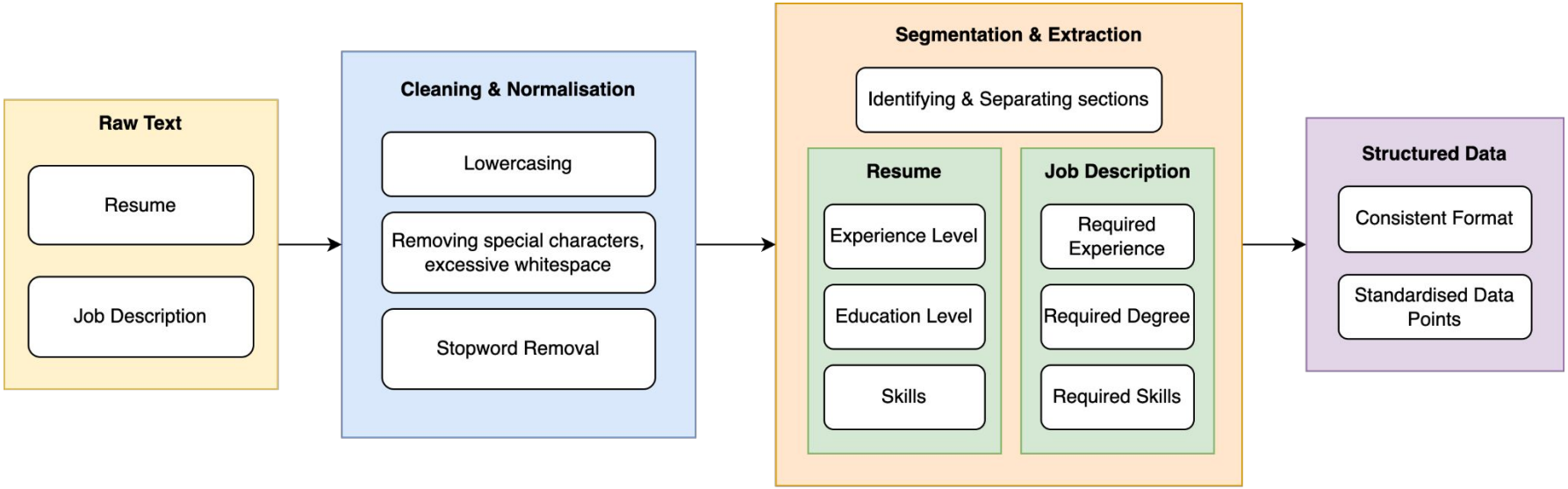
Data Overview

- **Focus:** STEM roles
- **Resumes** (1,000 rows)
  - Category (e.g., ETL Developer)
  - Raw Text
- **Job Descriptions** (143,000 rows)
  - Company Name
  - Job Title (e.g., Software Engineer)
  - Category (e.g., Java)
  - Raw Text
- **Data Sources:**
  - HuggingFace
  - Kaggle





Data Preprocessing



Matching Approaches

Input: Resume

Output: Top-N ranked job adverts

- **Baseline** (Rule-Based)
  - Skills overlap, experience, education, title
  - Composite weighted score
- **SBERT** (Contextual)
  - Embedding-based similarity
  - Fine-tuned on category-labeled data
- **Hybrid**
  - Combines rule-based + SBERT
  - Weighted sum of both methods

	Baseline (Rule-Based)	SBERT (Semantic)	Hybrid (Combined)
Interpretability	✓	✗	✓
Easy to Implement	✓	✗	✗
Training	✗	✓	✓
Contextual Understanding	✗	✓	✓
Domain Requirement	✗	✓	✓
Synonym Handling	✗	✓	✓
Noise Handling	✗	✓	✓
Weights Customisation	✓	✗	✓

Baseline (Rule-Based)

- Core Components:
  - **Skills Similarity:** Jaccard overlap
  - **Experience Match**: Linear Decay
  - **Education Match**: Filter
  - **Title Match**: Pre-defined Map

$$\begin{aligned}\text{Total Score} = {} & w_{\text{skills}} \times \text{SkillSim} \\ & + w_{\text{edu}} \times \text{EduMatch} \\ & + w_{\text{exp}} \times \text{ExpMatch} \\ & + w_{\text{title}} \times \text{TitleMatch}\end{aligned}$$

Resume	
current_level	junior
exp_years	1
education_type	undergraduate
degree	computer science
skills	[python, java, sql, r, pandas, pytorch, excel]
job_titles	data engineer

Job A	
job_level	entry
required_exp	2
education_requirement	undergraduate
degree_preferred	computer science
skills_required	[python, r, excel]
title	data engineer
match_score	0.72

Job B	
job_level	mid
required_exp	1
education_requirement	any
degree_preferred	data science
skills_required	[java, sql, pandas]
title	data analyst
match_score	0.65

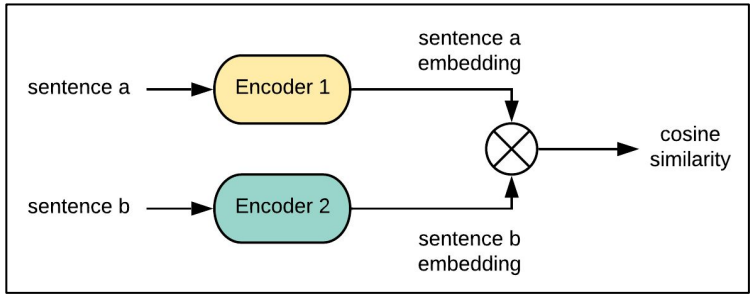
Job C	
job_level	senior
required_exp	3
education_requirement	postgraduate
degree_preferred	computer science
skills_required	[python, sql, pytorch, excel]
title	ml engineer
match_score	0.48

SBERT-Based Approach

What is SBERT?

Sentence
Bidirectional
Encoder
Representations from
Transformers

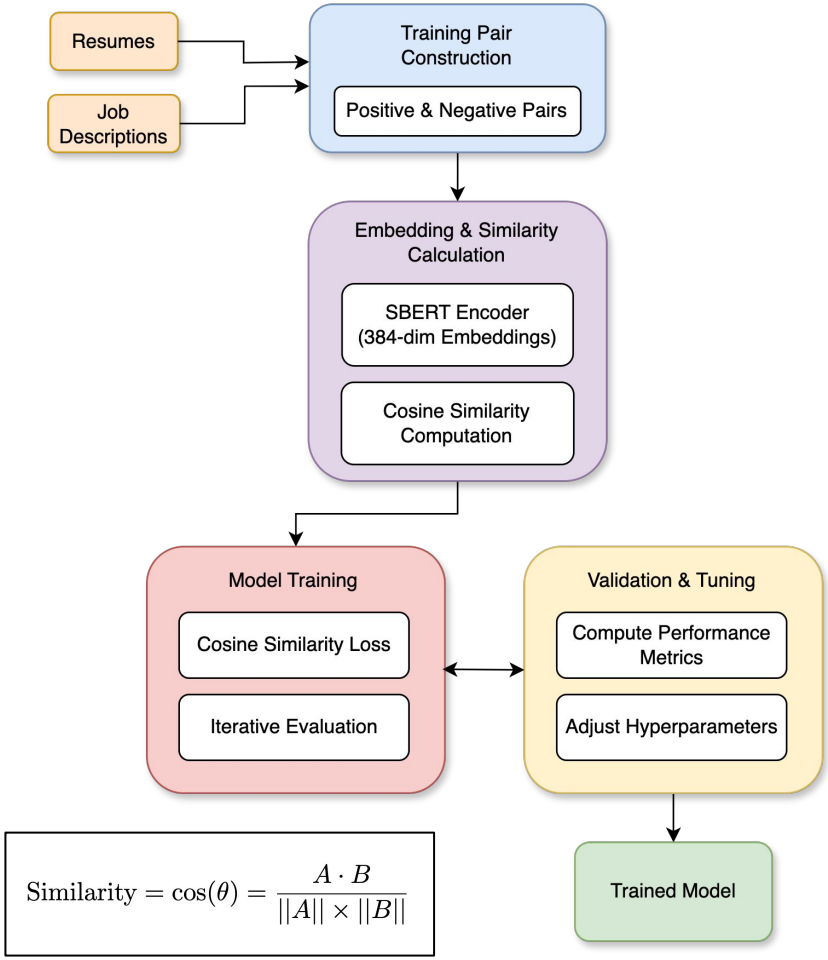
- Sentence-BERT (SBERT) is an extension of BERT optimised for **sentence similarity** tasks
- SBERT generates **dense vector embeddings** for entire sentences
- Compute **sentence similarities** (angle)




```python
from sentence_transformers import SentenceTransformer

# 1. Load a pretrained Sentence Transformer model
model = SentenceTransformer("all-MiniLM-L6-v2")

# The sentences to encode
sentences = [
    "The weather is lovely today.",
    "It's so sunny outside!",
    "He drove to the stadium.",
]

# 2. Calculate embeddings by calling model.encode()
embeddings = model.encode(sentences)
print(embeddings.shape)
# [3, 384]

# 3. Calculate the embedding similarities
similarities = model.similarity(embeddings, embeddings)
print(similarities)
# tensor([[1.0000, 0.6660, 0.1046],
#         [0.6660, 1.0000, 0.1411],
#         [0.1046, 0.1411, 1.0000]])
```

SBERT-Based Approach

- **Key Goal:** Capture semantic relationships
- **Training Data Construction:**
  - Category-based positive/negative pairs
    - **1:** Data Science ↔ Data & Analytics
    - **0:** Data Science ↔ HR
- **Embedding:** 384-dim dense vectors
- **Fine-Tuning Process:**
  - **Model:** SBERT
  - **Loss:** Cosine Similarity
  - **Settings:**
    - 80/20 split
    - Batch size = 16
    - 5 epochs



$$\text{Similarity} = \cos(\theta) = \frac{A \cdot B}{||A|| \times ||B||}$$

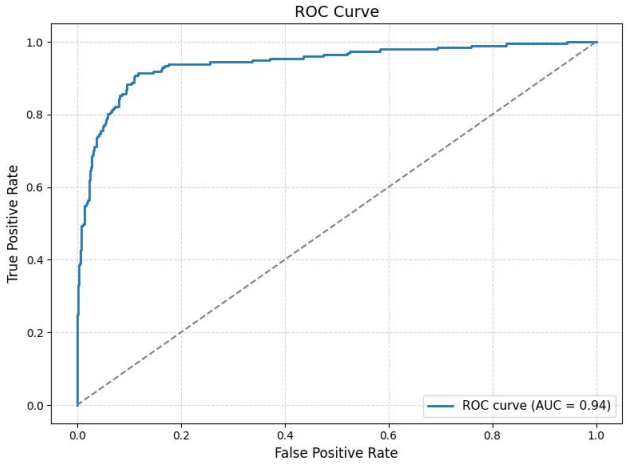
Evaluation – Metrics

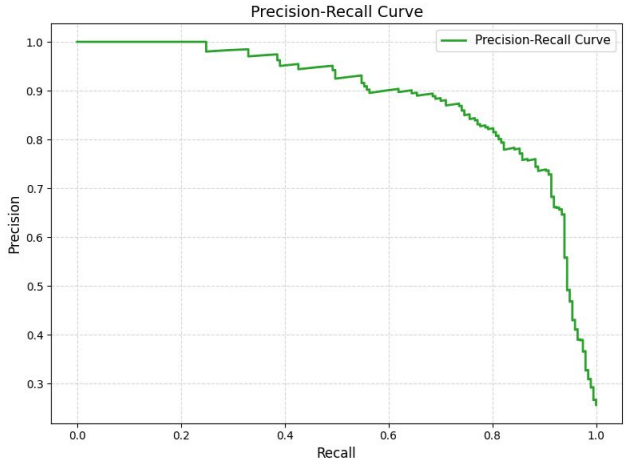
- **No Official Ground Truth → Synthetic Labels**
- **ROC Curve (AUC):**
  - ~0.94 indicates robust separability
- **Precision & Recall:**
  - Both high across classes

Classification Report

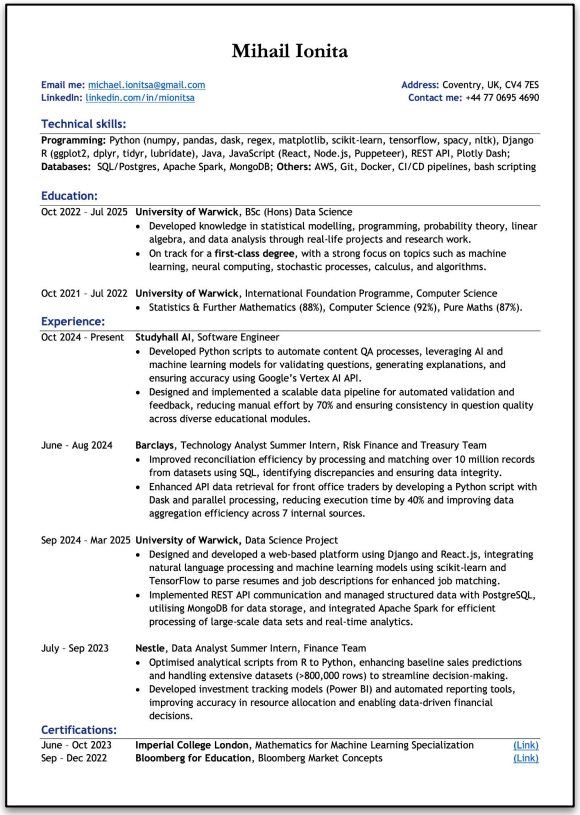
	Precision	Recall	F1-Score
Class 0	0.91	0.97	0.94
Class 1	0.88	0.71	0.79
Macro Avg	0.89	0.84	0.86
Weighted Avg	0.90	0.90	0.90

Accuracy: 0.901





Evaluation – Example



Mihail Ionita

Email me: michael.ionitsa@gmail.com
LinkedIn: linkedin.com/in/mionitsa

Address: Coventry, UK, CV4 7ES
Contact me: +44 77 0695 4690

Technical skills:

Programming: Python (numpy, pandas, dask, regex, matplotlib, scikit-learn, tensorflow, spacy, nltk), Django
R (ggplot2, dplyr, tidyr, lubridate), Java, JavaScript (React, Node.js, Puppeteer), REST API, Plotly Dash;
Databases: SQL/Postgres, Apache Spark, MongoDB; **Others:** AWS, Git, Docker, CI/CD pipelines, bash scripting

Education:

Oct 2022 – Jul 2025 **University of Warwick**, BSc (Hons) Data Science
- Developed knowledge in statistical modelling, programming, probability theory, linear algebra, and data analysis through real-life projects and research work.
- On track for a **first-class degree**, with a strong focus on topics such as machine learning, neural computing, stochastic processes, calculus, and algorithms.

Oct 2021 – Jul 2022 **University of Warwick**, International Foundation Programme, Computer Science
- Statistics & Further Mathematics (88%), Computer Science (92%), Pure Maths (87%).

Experience:

Oct 2024 – Present **Studyhall AI**, Software Engineer
- Developed Python scripts to automate content QA processes, leveraging AI and machine learning models for validating questions, generating explanations, and ensuring accuracy using Google's Vertex AI API.
- Designed and implemented a scalable data pipeline for automated validation and feedback, reducing manual effort by 70% and ensuring consistency in question quality across diverse educational modules.

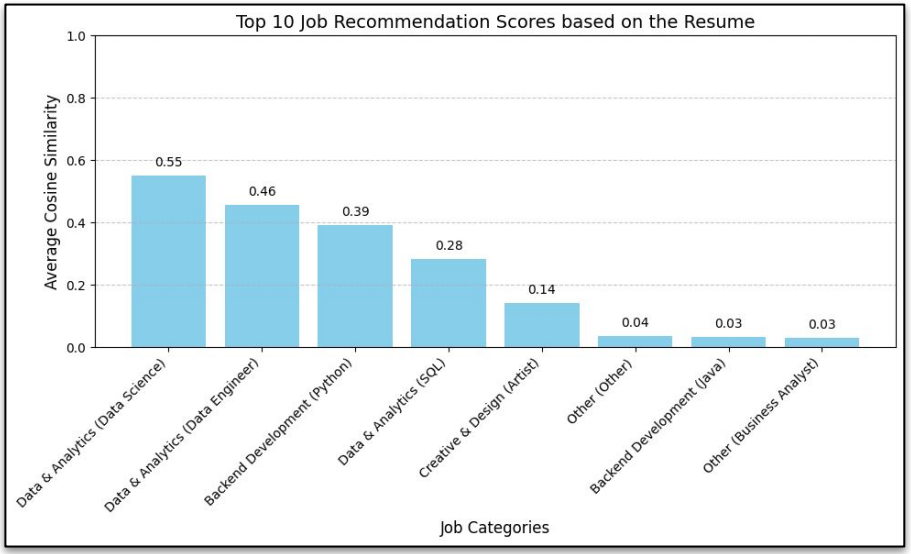
June – Aug 2024 **Barclays**, Technology Analyst Summer Intern, Risk Finance and Treasury Team
- Improved reconciliation efficiency by processing and matching over 10 million records from datasets using SQL, identifying discrepancies and ensuring data integrity.
- Enhanced API data retrieval for front office traders by developing a Python script with Dask and parallel processing, reducing execution time by 40% and improving data aggregation efficiency across 7 internal sources.

Sep 2024 – Mar 2025 **University of Warwick,** Data Science Project
- Designed and developed a web-based platform using Django and React.js, integrating natural language processing and machine learning models using scikit-learn and TensorFlow to parse resumes and job descriptions for enhanced job matching.
- Implemented REST API communication and managed structured data with PostgreSQL, utilising MongoDB for data storage, and integrated Apache Spark for efficient processing of large-scale data sets and real-time analytics.

July – Sep 2023 **Nestle**, Data Analyst Summer Intern, Finance Team
- Optimised analytical scripts from R to Python, enhancing baseline sales predictions and handling extensive datasets (>800,000 rows) to streamline decision-making.
- Developed investment tracking models (Power BI) and automated reporting tools, improving accuracy in resource allocation and enabling data-driven financial decisions.

Certifications:

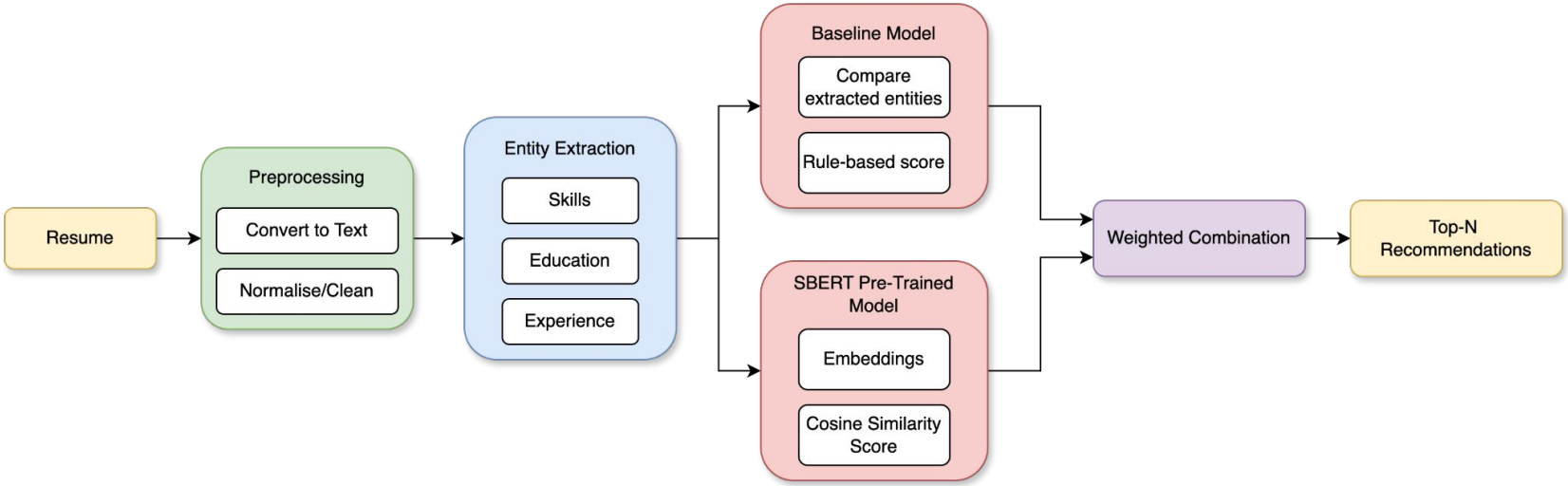
June – Oct 2023 **Imperial College London,** Mathematics for Machine Learning Specialization (Link)
Sep – Dec 2022 **Bloomberg for Education**, Bloomberg Market Concepts (Link)



Hybrid Approach

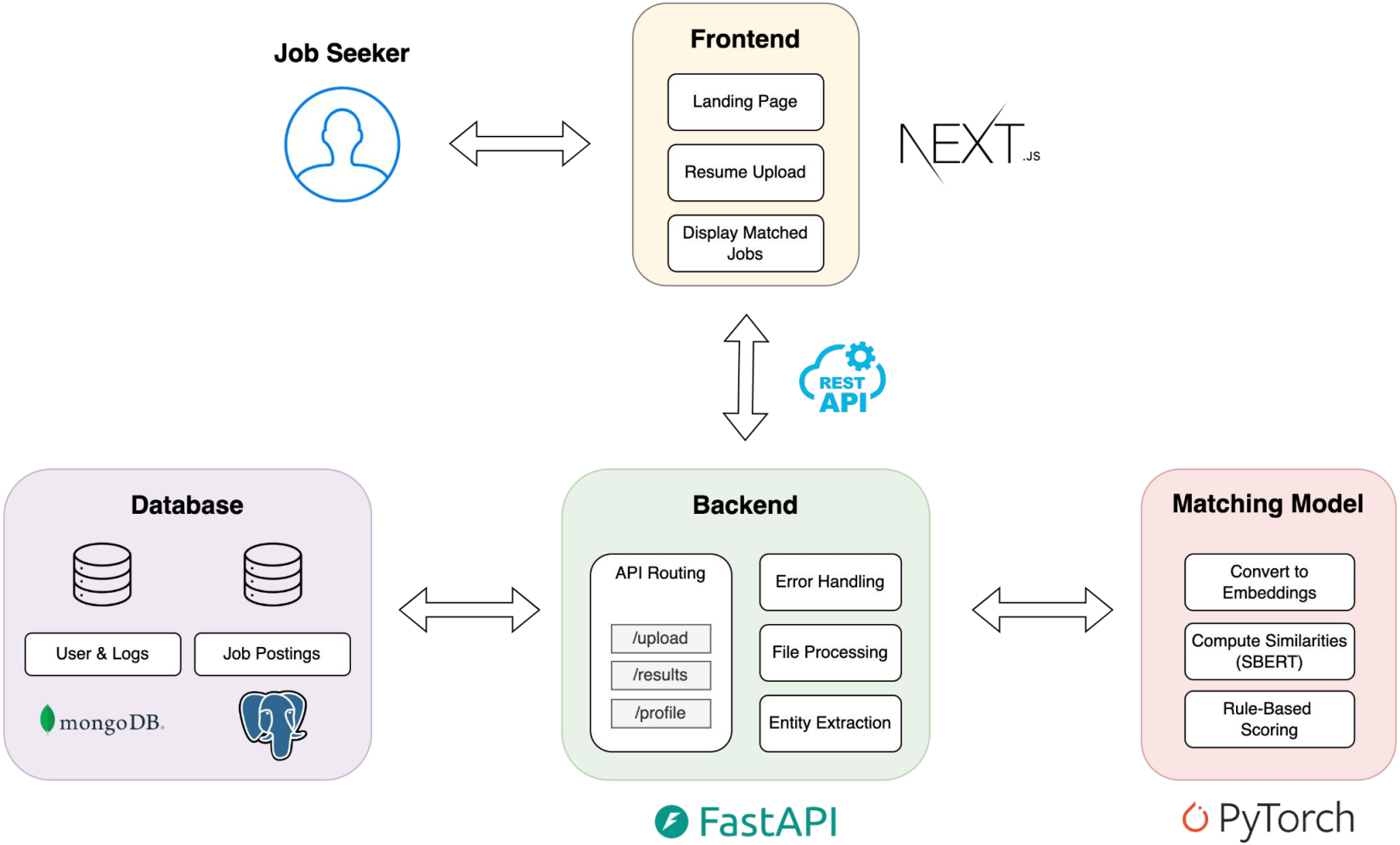
- **Why Hybrid?**
  - Leverages contextual understanding
  - Retains explicit rule-based checks

$$\text{Hybrid Score} = \alpha \times \text{SBERT Score} + (1-\alpha) \times \text{Rule-Based Score}$$



Section Outline

- ✓ Motivation & Project Objective
- ✓ Matching Model
- ➡ **Web Development**
  - Application Architecture
- Demo
- Limitations & Future Work
- Project Management
- Q&A


Job Seeker
Frontend
Landing Page
Resume Upload
Display Matched Jobs
NEXT.JS
REST API
Database
User & Logs
Job Postings
mongoDB.
Backend
API Routing
/upload
/results
/profile
Error Handling
File Processing
Entity Extraction
FastAPI
Matching Model
Convert to Embeddings
Compute Similarities (SBERT)
Rule-Based Scoring
PyTorch

Section Outline

- ✓ Motivation & Project Objective
- ✓ Matching Model
- ✓ Web Development
- ➡ **Demo**
  - Live App Demonstration
- Limitations & Future Work
- Project Management
- Q&A

Section Outline

- ✓ Motivation & Project Objective
- ✓ Matching Model
- ✓ Web Development
- ✓ Demo
- ➡ **Limitations & Future Work**
  - Contributions & Strengths
  - Limitations
  - Further Enhancements
- Project Management
- Q&A

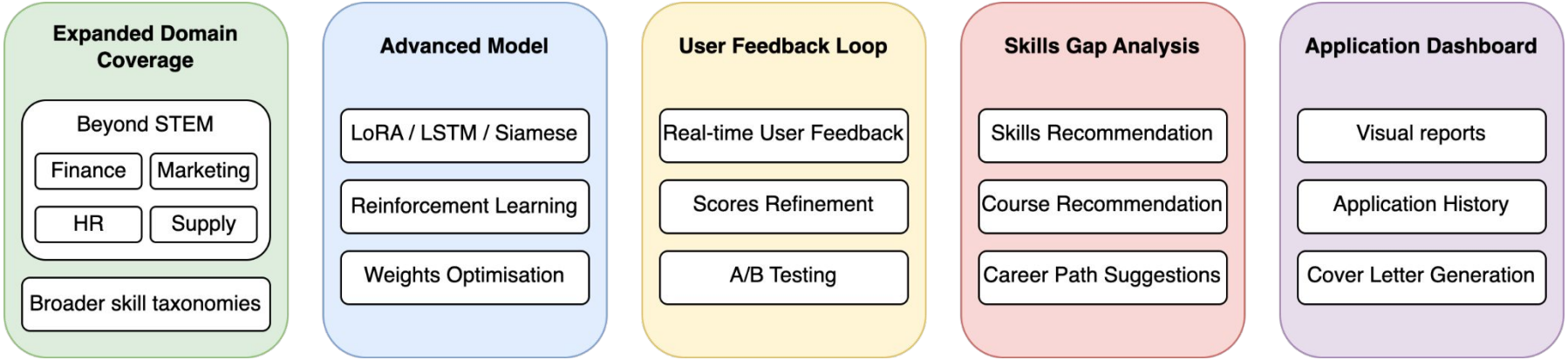
Contributions & Strengths

- **Enhanced Matching Accuracy**
  - Hybrid approach leverages both semantic embeddings (SBERT) and structured criteria
- **Improved Contextual Understanding**
  - Fine-tuned SBERT captures nuances beyond simple keyword matching
- **Integrated Pipeline**
  - End-to-end automatic system from resume upload to job recommendations

Limitations

- **Scope Restriction**
  - Focus primarily on STEM job roles and skill sets.
- **Synthetic Training Data**
  - Lack of real-world job-candidate pairings with ground-truth labels
- **Static Weighting Scheme**
  - Heuristic weights in the rule-based and hybrid component may not adapt to evolving job market trends

Future Work & Enhancements

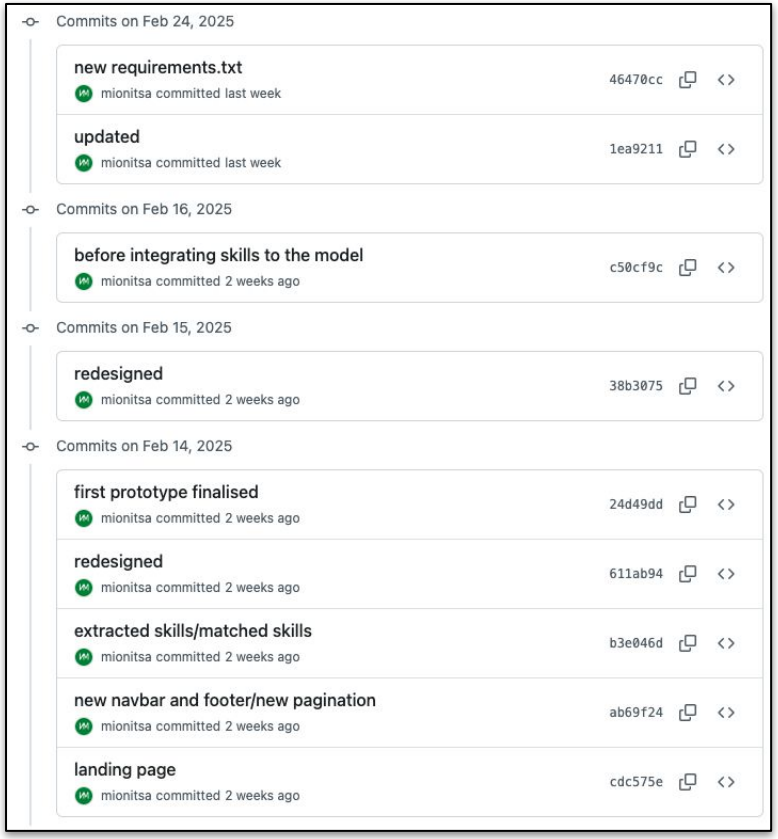


Section Outline

- ✓ Motivation & Project Objective
- ✓ Matching Model
- ✓ Web Development
- ✓ Demo
- ✓ Limitations & Future Work
- ➡ **Project Management**
  - Project and Development Organisation
  - Timeline
- Q&A

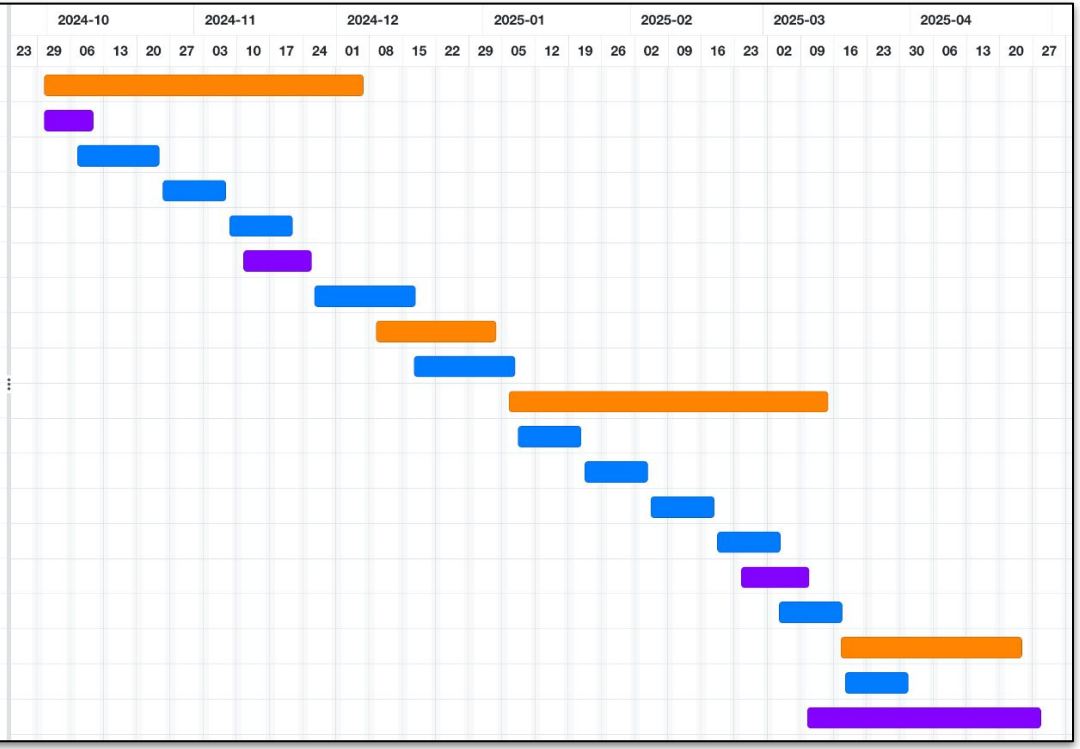
Project Organisation

- **Agile Methodology:**
  - 2–3 week sprints for iterative development
  - Flexibility for incorporating feedback & adjusting priorities
- **Version Control:**
  - Git-based repository (GitHub) for tracking changes
  - Cloud Storage (Google Colab)
- **Regular Meetings**
  - Weekly supervisor reviews with clear agendas
  - Continuous progress tracking
- **Deliverables:**
  - Iterative prototypes & incremental testing
  - Early MVP showcasing core functionalities



Timeline

Task Name	Start	End	Duration
Term 1	2024-09-30	2024-12-06	50 days
Project Specification Report	2024-09-30	2024-10-10	9 days
Initial Research and Literature Review	2024-10-07	2024-10-24	14 days
Requirements Gathering and Analysis	2024-10-25	2024-11-07	10 days
System Design and Initial Implementation	2024-11-08	2024-11-21	10 days
Progress Report Preparation	2024-11-11	2024-11-25	11 days
Implementation - Resume Parsing Module	2024-11-26	2024-12-17	16 days
Winter Break	2024-12-09	2025-01-03	20 days
Implementation - Job Description Parsing Mo…	2024-12-17	2025-01-07	16 days
Term 2	2025-01-06	2025-03-14	50 days
Implementation - Matching Algorithm	2025-01-08	2025-01-21	10 days
Implementation - User Interface Development	2025-01-22	2025-02-04	10 days
Integration and System Testing	2025-02-05	2025-02-18	10 days
User Feedback and Refinement	2025-02-19	2025-03-04	10 days
Oral Presentation Preparation	2025-02-24	2025-03-10	11 days
Implementation of Additional Features	2025-03-04	2025-03-17	10 days
Spring Break	2025-03-17	2025-04-24	29 days
Final Testing and Quality Assurance	2025-03-18	2025-03-31	10 days
Final Report Writing	2025-03-10	2025-04-28	36 days



References

[1] Kaya, M., & Bogers, T. (2021). Effectiveness of job title-based embeddings on résumé to job ad recommendation. In Proceedings of the Workshop on Recommender Systems for Human Resources (RecSys in HR 2021) co-located with the 15th ACM Conference on Recommender Systems. CEUR Workshop Proceedings, 2967, 1-8. Available at: https://ceur-ws.org/Vol-2967/paper_5.pdf

[2] Zhao, J., Wang, J., Sigdel, M., Zhang, B., Hoang, P., Liu, M., & Korayem, M. (2021). Embedding-based recommender system for job to candidate matching on scale. In Proceedings of the 2nd International Workshop on Industrial Recommendation Systems (IRS 2021) co-located with the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining. Available at: https://irsworkshop.github.io/2021/publications/IRS2021_paper_6.pdf

[3] Yu, X., Zhang, J., & Yu, Z. (2024). ConFit: Improving resume-job matching using data augmentation and contrastive learning. arXiv preprint arXiv:2401.16349. Available at: https://arxiv.org/html/2401.16349v1

[4] Singla, P., & Verma, V. (2025). An intelligent job recommendation system based on semantic embeddings and machine learning. ResearchGate. Available at: https://www.researchgate.net/publication/388741604_An_Intelligent_Job_Recommendation_System_based_on_Semantic_Embeddings_and_Machine_Learning

[5] Nigam, A., Roy, A., Saxena, A., & Singh, H. (2020). Job recommendation through progression of job selection. arXiv preprint arXiv:1905.13136. Available at: https://arxiv.org/abs/1905.13136

Datasets:

[6] Drushchak, N., & Romanyshyn, M. (2024). Introducing the Djinni Recruitment Dataset: A corpus of anonymized CVs and job postings. In Proceedings of the Third Ukrainian Natural Language Processing Workshop (UNLP) @ LREC-COLING 2024 (pp. 8-13). Torino, Italia: ELRA and ICCL. Available at: https://aclanthology.org/2024.unlp-1.2

[7] Dutta, G. (2020). Resume dataset. Kaggle. Available at: https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset

Section Outline

- ✓ Motivation & Project Objective
- ✓ Matching Model
- ✓ Web Development
- ✓ Demo
- ✓ Limitations & Future Work
- ✓ Project Management
- ➡ **Q&A**